同济大学

TONGJI UNIVERSITY

人工智能课程设计

课题名称	多算法的机器人避障寻址问题
学　　院	国豪书院
专　　业	数据科学与大数据技术
学生姓名	费凡
学　　号	2454259
指导教师	苗夺谦，张奇
日　　期	2026.4.20

多算法的机器人避障寻址问题

摘 要

随着机器人技术的快速发展，自主导航已成为智能机器人领域的核心研究内容。路径规划作为导航系统的关键环节，直接影响机器人在复杂环境中的作业效率与安全性。本文针对静态栅格地图下的机器人避障寻径问题，设计并实现了一套多算法对比实验平台，涵盖经典搜索算法（BFS、A、Dijkstra）、启发式算法（人工势场法 APF、Bug2）、进化算法（遗传算法 GA）以及多种强化学习算法（Q-Learning、SARSA、Double Q-Learning、深度 Q 网络 DQN）。通过在三类不同复杂度的地图上进行对比实验，从成功率、路径长度、搜索耗时和算法特点等维度对上述算法进行了系统性评估。实验结果表明，A*算法在保证最优性的同时具有较高的搜索效率，适用于静态已知环境；强化学习算法在复杂迷宫中具备一定的学习能力，但训练耗时较长且稳定性有待提升；APF 与 Bug2 算法虽实现简单，但在某些场景中易陷入局部最优而找不到目标。本文工作为机器人路径规划算法的选择与优化提供了参考依据，并构建了可视化的算法演示平台，有助于直观理解不同算法的搜索机制。

关键词：路径规划，移动机器人，避障寻址，A*算法，强化学习

装 订 线

目 录

1 引 言 1
1.1 研究背景与意义 1
1.2 国内外研究现状 1
1.3 本文所作的工作 1
2 算法原理 3
2.1 栅格地图建模 3
2.2 经典算法 3
2.2.1 广度优先搜索 BFS 3
2.2.2 Dijkstra 算法 4
2.2.3 A*算法 5
2.2.4 A*与 BFS 的详细比较 8
2.3 启发式与进化算法 8
2.3.1 人工势场法（APF） 8
2.3.2 Bug2 算法 9
2.3.3 遗传算法（GA） 11
2.4 强化学习算法 12
2.4.1 Q-Learning 与 SARSA 12
2.4.2 Double Q-Learning 14
2.4.3 深度 Q 网络与 DQN 14
3 实验设计与实现 16
3.1 实验环境与工具 16
3.2 地图设计 16
3.3 算法实现框架 17
3.4 可视化方案 18
4 实验结果与分析 20
4.1 原始地图 20

装 订 线

4.2 U 形陷阱地图.....21
4.3 自定义迷宫地图.....22
4.4 算法综合对比分析.....23
4.5 深入讨论.....24
5 结论和展望.....25
5.1 结论.....25
5.2 展望.....25
参考文献.....27

装 订 线

1 引言

1.1 研究背景与意义

移动机器人已广泛应用于工业制造、仓储物流、家庭服务、星际探测等众多领域。自主导航能力是移动机器人智能化的核心体现，而路径规划作为导航系统的“大脑”，负责在具有障碍物的环境中寻找一条从起点到终点的安全、高效的运动轨迹[1]。路径规划算法的优劣直接决定了机器人的任务执行效率与运行安全性。

按照对环境信息的掌握程度，路径规划可分为全局规划与局部规划。全局规划依赖于环境的先验地图，旨在寻找全局最优路径；局部规划则侧重于实时避障，动态调整运动轨迹。本文聚焦于静态栅格地图下的全局路径规划问题，此类问题具有明确的数学模型，便于算法性能的定量对比与分析。近年来，除传统的图搜索算法外，基于启发式规则、进化计算及强化学习的路径规划方法层出不穷，各有优劣。系统性地对比这些算法在不同场景下的表现，对于工程实践中算法选型具有重要的指导意义。

1.2 国内外研究现状

路径规划的研究历史悠久。1959 年，Dijkstra 提出了求解单源最短路径的经典算法[2]。1968 年，Hart 等人提出的 A*算法[3]引入了启发式函数，极大地提升了搜索效率，至今仍是应用最广泛的路径规划算法之一。广度优先搜索（BFS）作为图遍历的基础算法，在无权图中可保证最短路径，但搜索效率较低。

针对复杂环境，研究者提出了多种启发式及仿生算法。Khatib 于 1986 年提出人工势场法（APF）[4]，通过虚拟力场引导机器人运动，实现简单但易陷入局部极小。Bug 系列算法[5]通过“向目标直行—沿障碍绕行”的切换策略，在未知环境中具有一定的鲁棒性。遗传算法（GA）[6]模拟自然进化过程，通过选择、交叉、变异操作优化路径，适用于多约束条件下的全局寻优。

近年来，强化学习（RL）在路径规划领域展现出巨大潜力。Watkins 提出的 Q-Learning 算法[7]无需环境模型，通过与环境交互学习最优策略。SARSA、Double Q-Learning 等变体进一步提升了学习稳定性[8]。深度强化学习的兴起使得端到端的路径规划成为可能，深度 Q 网络（DQN）[9]利用神经网络拟合 Q 值函数，能够处理高维状态空间。

当前研究多集中于单一算法的改进，缺乏对多类算法在同一基准下的横向对比。本文构建统一的实验平台，将经典搜索、启发式、进化及强化学习算法纳入同一框架进行可视化对比，以期为算法选型提供直观参考。

1.3 本文所作的工作

本文围绕“机器人避障寻址”问题，完成了以下工作：

（1）**搭建多算法路径规划实验平台。**基于 Python 语言和 Gym 接口规范，设计了统一的栅格环境接口，集成了 BFS、A*、Dijkstra、APF、Bug2、GA、Q-Learning、SARSA、Double Q-Learning、

装 订 线

DQN 共十种路径规划算法。

（2）**设计三组不同复杂度的测试地图。**包括简单障碍的原始地图、U 形陷阱地图和自定义复杂迷宫地图，以全面评估算法的适应性。

（3）**实现搜索过程与最终路径的动态可视化。**利用 Matplotlib 动画模块，对搜索过程中已访问节点、边界节点及最终路径进行实时渲染，直观展示算法行为。

（4）**开展系统性对比实验。**从成功率、路径长度、搜索耗时、算法原理特点等维度对十种算法进行深入分析，总结各算法的适用场景与局限性。

装 订 线

2 算法原理

本章对本文所实现的十种路径规划算法进行系统性的原理阐述。按照算法类型，分为经典搜索算法（BFS、Dijkstra、A*）、启发式与进化算法（APF、Bug2、GA）以及强化学习算法（Q-Learning、SARSA、Double Q-Learning、DQN）三大类。对每种算法，将详细介绍其核心思想、数学基础、执行流程，并分析其优缺点与适用场景。

2.1 栅格地图建模

本文采用二维栅格地图对环境进行离散化建模，这是移动机器人路径规划中最常用的环境表示方法。

（1）栅格地图：设环境被划分为 M 行 N 列的均匀栅格，用一个矩阵 G(大小为 M×N) 表示，矩阵元素取值为 0 或 1。其中：

G(i,j) = 0 表示栅格 (i,j) 为自由区域，机器人可通行；

G(i,j) = 1 表示栅格 (i,j) 为障碍区域，机器人不可通行。

（2）动作空间：机器人在栅格地图中的运动约束为四连通方向，即每次移动可选择上、下、左、右四个方向之一，分别对应坐标变化量：

$\Delta \in \{ (-1,0), (1,0), (0,-1), (0,1) \}$

（3）路径与代价：一条从起点 S 到终点 G 的路径定义为一个栅格序列 P = [p0, p1, …, pk]，其中 p0 = S，pk = G，且对任意 $i \in [0, k-1]$，p(i+1) 为 pi 的四连通邻居且均为自由栅格。每次移动的代价统一设为 1，路径长度定义为 |P| = k（即经过的栅格总数减 1）。

基于上述模型，机器人避障寻径问题转化为：在矩阵 G 中寻找一条从 S 到 G 的路径 P，使得 |P| 最小且所有中间节点均为自由栅格。

2.2 经典算法

经典搜索算法将路径规划问题视为图搜索问题，通过系统性地探索状态空间来寻找最优解。本节详细介绍 BFS、Dijkstra 和 A* 三种算法的原理与实现。

2.2.1 广度优先搜索 BFS

1. 算法原理

广度优先搜索（Breadth-First Search，BFS）是一种图遍历算法，其核心思想是按层扩展：从起点开始，先访问所有距离为 1 的节点，再访问距离为 2 的节点，依此类推，直到到达目标节点。在无权图（或所有边权相等）中，BFS 首次访问到目标节点时所经过的路径即为最短路径。

BFS 使用一个队列（Queue）数据结构来维护待扩展的节点集合，遵循“先进先出”（FIFO）原则，保证了节点按照距离起点的层数顺序被处理。

2. 算法流程

输入：栅格地图 grid，起点 start，终点 goal

输出：最短路径 path 或 None（不可达）

装 订 线

```
1.  初始化队列 Q，将起点 start 入队
2.  初始化集合 visited = {start}，记录已访问节点
3.  初始化字典 parent = {}，记录每个节点的父节点
4.  while Q 非空 do
5.      current ← Q 出队首元素
6.      if current == goal then
7.          通过 parent 回溯重建路径并返回
8.      end if
9.      for each neighbor ∈ get_neighbors(current) do
10.         if neighbor ∉ visited then
11.             visited.add(neighbor)
12.             parent[neighbor] ← current
13.             Q 入队 neighbor
14.         end if
15.     end for
16. end while
17. return None    // 队列为空，目标不可达
```

3. 算法分析

（1）完备性：若存在从起点到终点的路径，BFS 一定能找到。

（2）最优性：在边权相等的栅格地图中，BFS 保证找到最短路径（步数最少）。

（3）时间复杂度：每个节点最多入队一次、出队一次，每条边被检查一次。对于 M×N 的栅格地图，节点数为 O(MN)，边数为 O(MN)，故时间复杂度为 O(MN)。

（4）空间复杂度：队列和 visited 集合最多存储所有节点，空间复杂度为 O(MN)。

4. 局限性分析

BFS 的主要局限在于盲目搜索——它不考虑目标的方向信息，在开阔区域会向四周均匀扩散，导致搜索空间急剧膨胀。当起点与终点距离较远时，BFS 需要探索大量无关节点，内存消耗大、效率低。此外，BFS 仅适用于边权相等的图，若存在不同代价的地形（如草地、沙地），BFS 无法直接处理。

5. 改进思路

（1）引入双向搜索，从起点和终点同时扩展，相遇时终止，可将搜索空间减半；

（2）结合启发式信息（即演化为 A* 算法）引导搜索方向。

2.2.2 Dijkstra 算法

1. 算法原理

装 订 线

Dijkstra 算法由荷兰计算机科学家 Edsger W. Dijkstra 于 1959 年提出，用于求解带权图中单源最短路径问题。其核心思想是贪心策略：每次从尚未确定最短距离的节点集合中，选择当前距离起点最近的节点进行扩展，并尝试更新其邻居的距离估计值。

Dijkstra 算法使用优先队列维护待扩展节点，按照节点当前的累计代价 g(n) 排序，确保每次扩展的都是已知距离最小的节点。

2. 算法流程

Dijkstra 算法使用优先队列维护待扩展节点，每次取出当前实际代价 g(n)最小的节点进行扩展，并尝试更新邻居节点的代价值，其详细流程与 BFS 类似，区别仅在于使用优先队列代替普通队列。

3. 与 BFS 的关系

在本文的栅格地图中，所有可通行的边代价均为 1，此时 Dijkstra 算法的行为与 BFS 完全等价——每次扩展的节点都是距离起点步数最少的节点。但 Dijkstra 的实现框架更具普适性，可无缝扩展至带权地图（例如不同地形具有不同通行代价）。当边权相等时，优先队列退化为普通队列，算法退化为 BFS。

4. 算法分析

（1）完备性与最优性：Dijkstra 算法是完备的，且在所有边权非负的情况下保证找到最短路径。

（2）时间复杂度：使用二叉堆实现的优先队列，每次入队/出队操作的时间为 O(log V)，其中 $V = M \times N$ 为节点数。总时间复杂度为 $O((V+E)\log V) = O(MN \log(MN))$。

（3）空间复杂度：$O(V) = O(MN)$。

5. 局限性分析

Dijkstra 算法虽然能够处理带权图，但依然是无信息搜索——它在所有方向上均匀扩展，不利用任何关于目标位置的信息，因此在大型地图中效率较低。此外，若图中存在负权边，Dijkstra 算法将失效，需改用 Bellman-Ford 等算法。

2.2.3 A*算法

1. 算法原理

A* 算法由 Hart、Nilsson 和 Raphael 于 1968 年提出，是 Dijkstra 算法的启发式改进版本。它在 Dijkstra 的基础上引入了启发式函数 h(n)，用于估计从当前节点 n 到目标节点的代价。A* 的评价函数定义为：

$f(n) = g(n) + h(n)$

其中：

g(n) 为从起点到节点 n 的实际代价（与 Dijkstra 一致）；

h(n) 为从节点 n 到目标的启发式估计代价。

A* 每次从优先队列中选取 f(n) 值最小的节点进行扩展，从而实现有导向的搜索——优先探

装 订 线

索那些“看起来更靠近目标”的节点。

2. 启发式函数的选择

启发式函数 h(n) 的设计直接影响 A* 算法的性能与最优性。常用的启发式函数包括：

（1）曼哈顿距离：适用于四连通栅格地图。

$$h(n) = | x_n - x_{goal} | + | y_n - y_{goal} |$$

（2）欧几里得距离：适用于可任意方向移动的场景。

$$h(n) = \sqrt{(x_n - x_{goal})^2 + (y_n - y_{goal})^2}$$

（3）切比雪夫距离：适用于八连通栅格地图。

$$h(n) = \max(| x_n - x_{goal} |, | y_n - y_{goal} |)$$

可纳性与一致性是保证 A* 最优性的关键条件：

（1）可纳性：对任意节点 n，h(n)≤h*(n)，*其中* $h^*(n)$ 为实际最短代价。即启发式函数不高估实际代价。

（2）一致性：对任意节点 n 及其邻居 n'，满足 h(n) ≤ cost(n, n') + h(n')。

曼哈顿距离在四连通栅格地图中既是可纳的也是一致的，因此 A* 在此条件下保证找到最短路径。

3. 算法流程

输入：栅格地图 grid，起点 start，终点 goal

输出：最短路径 path 或 None

1. 初始化优先队列 PQ，将 (f(start), start) 入队
2. 初始化字典 g_score，g_score[start] ← 0，其余为无穷大
3. 初始化字典 f_score，f_score[start] ← h(start)
4. 初始化字典 parent = {}
5. 初始化集合 open_hash = {start} // 快速查找节点是否在开放集中
6. 初始化集合 closed = {}
7. while PQ 非空 do
8. _, current ← PQ 弹出最小 f 值元素
9. if current ∉ open_hash then continue
10. open_hash.remove(current)
11. if current == goal then
12. 通过 parent 回溯重建路径并返回
13. end if
14. closed.add(current)
15. for each neighbor ∈ get_neighbors(current) do
16. if neighbor ∈ closed then continue

装 订 线

```
17.             tentative_g ← g_score[current] + 1
18.             if neighbor ∉ open_hash or tentative_g < g_score[neighbor] then
19.                 parent[neighbor] ← current
20.                 g_score[neighbor] ← tentative_g
21.                 f_score[neighbor] ← tentative_g + h(neighbor)
22.                 if neighbor ∉ open_hash then
23.                     PQ 入队 (f_score[neighbor], neighbor)
24.                     open_hash.add(neighbor)
25.                 end if
26.             end if
27.         end for
28. end while
29. return None
```

4. 算法分析

（1） 最优性与完备性：当 h(n) 满足可纳性时，A* 保证找到最短路径；当 h(n) 满足一致性时，A* 保证在扩展节点时即确定其最短距离，且不会重复扩展。

（2）时间复杂度：最坏情况下（如 h(n)=0，退化为 Dijkstra），时间复杂度为 O(MN log(MN))。实际运行中，良好的启发式函数可大幅减少扩展节点数。

（3） 空间复杂度：与 Dijkstra 相同，为 O(MN)。

5. A* 与 BFS/Dijkstra 的比较

特性	BFS	Dijkstra	A*
搜索策略	盲目、按层	盲目、按实际代价	启发式、按估计总代价
最优性保证	边权相等时	非负边权时	h(n) 可纳时
扩展节点数	最多	与 BFS 相同	显著少于前两者
适用场景	无权网格	带权网格	带权网格（有启发信息）
启发式依赖	无	无	强依赖 h(n) 质量

6. 局限性分析与改进点

A* 的主要局限性包括：

（1）启发式函数依赖：若 h(n) 设计不佳（如过于乐观或悲观），可能导致搜索效率下降甚至失去最优性。

（2）内存消耗大：在大规模地图中，A* 仍需维护庞大的 open 和 closed 集合，可能引发内存溢出。

（3）动态环境适应性差：A* 假设环境静态已知，若障碍物动态变化，需重新规划。

改进方向包括：

（1）加权 A（*Weighted A*）：$f(n) = g(n) + \varepsilon \cdot h(n)$，通过调节权重 ε 在最优性与搜索速度间权

装 订 线

衡。

（2）IDA（*Iterative Deepening A*）：采用深度优先的迭代加深策略，大幅降低内存占用。

（3）*D** Lite：针对动态环境设计的增量式启发式搜索算法。

2.2.4 A*与 BFS 的详细比较

1. 搜索策略

BFS 是一种盲目搜索方法，它按照节点距离起点的层数严格逐层向外扩展，使用普通队列（FIFO）管理待扩展节点，扩展顺序与目标方向无关，呈现出“水波式”的均匀扩散特征。

A* 则是一种启发式搜索方法，通过引入评价函数 f(n)=g(n)+h(n) 将目标方向信息融入搜索过程，其中 g(n) 为已走过的实际代价，h(n) 为启发式估计代价。A* 使用优先队列每次选取 f 值最小的节点优先扩展，使得搜索过程具有明显的方向偏好，倾向于向目标位置延伸。当启发式函数 h(n) 恒为零时，A* 退化为 Dijkstra 算法（在等权图中等价于 BFS）；当 h(n) 能够准确估计剩余代价时，A* 几乎直线冲向目标，扩展节点数达到理论下限。

2. 时空复杂度

BFS 的时间复杂度为 O(M×N)，使用队列操作的单次代价为 O(1)，运行速度很快，但扩展节点数与最短路径长度 L 的平方成正比，在开阔区域中会探索大量无关节点。

A* 因采用优先队列，单次入队出队操作的时间为 O(log(M×N))，单步代价略高于 BFS，但由于启发式引导，其实际扩展节点数远小于 BFS。空间复杂度二者均为 O(M×N)，但由于 A* 实际扩展节点数更少，其开放集和关闭集的平均规模也低于 BFS，内存占用更小。

3. 适用场景与局限性

BFS 实现极其简单，无需设计启发式函数，非常适合小规模地图，但其在带权图中无法直接应用，且面对大规模开阔地图时效率急剧下降。

A* 通过启发式函数实现了搜索效率的大幅跃升，是静态全局路径规划中兼顾最优性与效率的首选方案，同时可自然扩展至带权栅格场景。但 A* 的性能高度依赖启发式函数的设计质量。若 h(n) 估计过于乐观，扩展节点数可能增加；若 h(n) 过高估计，则可能丧失最优性。此外，A* 在动态环境中需要频繁重规划，此时更适合采用 D* Lite 等增量式搜索算法。

2.3 启发式与进化算法

2.3.1 人工势场法（APF）

1. 算法原理

人工势场法（Artificial Potential Field，APF）由 Khatib 于 1986 年提出，是一种基于虚拟力场的局部路径规划方法。其基本思想是：在机器人的工作空间中构造一个虚拟势场，目标点产生引力场，障碍物产生斥力场。机器人在合力作用下向势能最低的方向运动，最终抵达目标。

引力场函数通常定义为：

装 订 线

$$U_{att}(q) = \frac{1}{2}\xi \cdot d^2(q, q_{goal})$$

斥力场函数通常定义为（以单个障碍物为例）：

$$U_{rep}(q) = \begin{cases} \frac{1}{2}\eta\left(\frac{1}{d(q,q_{obs})} - \frac{1}{\rho_0}\right)^2, & d(q,q_{obs}) \le \rho_0 \\ 0, & d(q,q_{obs}) > \rho_0 \end{cases}$$

其中 η 为斥力增益系数，ρ0 为障碍物的影响范围。对应的斥力为：

$$F_{rep}(q) = \begin{cases} \eta\left(\frac{1}{d} - \frac{1}{\rho_0}\right)\frac{1}{d^2}\frac{\partial d}{\partial q}, & d \le \rho_0 \\ 0, & d > \rho_0 \end{cases}$$

机器人所受合力为 F_total = F_att + Σ F_rep，运动方向由合力的方向决定。

2. 算法流程

（1）预计算障碍距离场与梯度；

（2）从起点迭代计算合力；

（3）将合力方向离散为四连通方向并更新位置；

（4）检测是否到达目标、撞墙或陷入震荡。其中震荡检测通过记录最近 10 步位置变化范围实现，若范围小于 3 栅格则判定为局部极小并退出。

3. 优缺点与局限性

（1）优点：计算简单、实时性好，规划出的路径平滑，适合动态避障。

（2）缺点：

① 局部极小问题：在 U 形障碍或狭窄通道中，引力与斥力可能相互抵消，导致机器人停止或震荡。

② 目标不可达问题：当目标附近存在障碍物时，斥力可能大于引力，机器人无法抵达目标。

③ 参数敏感：引力/斥力增益系数及影响范围的选择对规划效果影响显著。

4. 改进措施

本文在 APF 实现中引入两项改进：

（1）震荡检测：记录最近 10 步的位置，若变化范围小于 3 个栅格，判定为陷入局部极小，返回规划失败。

（2）四连通映射：将连续合力方向离散化为上下左右之一，确保输出路径符合栅格移动约束。

2.3.2 Bug2 算法

1. 算法原理

Bug2 算法属于 Bug 系列算法，适用于未知环境中的局部路径规划。其核心规则为：

装 订 线

（1）维护一条从起点指向终点的虚拟直线（称为 M-line）。

（2）机器人首先沿 M-line 向目标直行（Motion-To-Goal，MTG 模式）。

（3）遇到障碍物时，切换为沿墙跟随（Wall-Following，WF 模式），紧贴障碍物边界运动。

（4）当沿墙过程中再次遇到 M-line 上且比初次撞墙点更接近目标的点时，脱离墙模式，恢复 MTG 模式。

2. 算法流程

输入：栅格地图 grid，起点 start，终点 goal

输出：路径 path 或 None

```
1. curr ← start；  path ← [curr]；  visited ← {curr}
2. mode ← "MTG"；  hit_point ← None
3. steps ← 0；  max_steps ← 2000
4. while steps < max_steps do
5.       if curr == goal then return path
6.       if mode == "MTG" then
7.             计算向目标的四连通步进方向 step
8.             next ← (curr.x + step.x, curr.y + step.y)
9.             if next 可行 then
10.                  curr ← next；  更新 path 和 visited
11.            else
12.                  mode ← "WF"；  hit_point ← curr
13.            end if
14.      else   // mode == "WF"
15.            neighbors ← 获取当前节点的所有可行邻居
16.            if neighbors 为空 then return None
17.            next ← 优先选择未访问过的邻居，否则选择第一个
18.            curr ← next；  更新 path 和 visited
19.            if hit_point ≠ None and curr 在 M-line 上 then
20.                  if 距离 goal 的距离 < 距离 hit_point 的距离 - 0.5 then
21.                        mode ← "MTG"；  hit_point ← None
22.                  end if
23.            end if
24.      end if
25.      steps ← steps + 1
```


装 订 线

26. end while

27. return None

3. 算法分析

（1）完备性：在任意有限、连通的二维环境中，Bug2 保证找到一条到达目标的路径（若存在）。

（2）路径质量：Bug2 产生的路径通常不是最优的，可能存在大量绕行。

（3）M-line 依赖：Bug2 的性能与 M-line 的选取密切相关。若 M-line 与障碍物边界交点极少，可能导致长时间沿墙无效移动。

4. 局限性

（1）效率低：在复杂迷宫中，Bug2 可能要遍历整个障碍物边界才能找到离开点。

（2）死锁风险：在非凸障碍或零距离接触场景中，简单的沿墙规则可能会死循环。

2.3.3 遗传算法（GA）

1. 算法原理

遗传算法（Genetic Algorithm，GA）是一种模拟自然选择和遗传机制的全局优化算法。本文将路径规划问题建模为组合优化问题，通过进化操作搜索最优路径。

个体编码：每条路径由起点、终点及若干中间路径点（waypoint）构成。个体表示为一个坐标序列 [S, w1, w2, …, wk, G]。中间路径点随机生成于自由栅格区域。

适应度函数：评估个体的优劣。适应度函数综合考虑：路径总长度（欧氏距离累加）；碰撞惩罚（若连接路径点的直线穿过障碍物，施加高额惩罚）；终点接近奖励（鼓励个体末端靠近目标）。

适应度函数定义为：

$$Fitness(P) = \frac{1}{Length(P) + \lambda_1 \cdot Collision(P) + \lambda_2 \cdot DistToGoal(P) + \epsilon}$$

遗传算子：

（1）选择：锦标赛选择（Tournament Selection）。随机选取 k 个个体，适应度最高者进入下一代。

（2）交叉：单点交叉。交换两个父代个体的部分中间路径点片段。

（3）变异：以一定概率随机改变个体中的一个中间路径点坐标。

（4）精英保留：每代适应度最高的若干个体直接保留至下一代。

2. 算法流程

输入：栅格地图 grid，起点 start，终点 goal

输出：最优路径 best_path

1. 种群初始化：随机生成 pop_size 个个体
2. best_individual ← None； best_fitness ← -∞

装 订 线

3. for gen = 1 to max_gen do
4. 计算每个个体的适应度
5. 更新全局最优个体
6. 精英保留：适应度最高的 elite_size 个个体直接进入下一代
7. 新种群 ← 精英个体
8. while 新种群规模 < pop_size do
9. 父代 1 ← 锦标赛选择(种群, 适应度)
10. 父代 2 ← 锦标赛选择(种群, 适应度)
11. 子代 1, 子代 2 ← 交叉(父代 1, 父代 2)
12. 变异(子代 1)； 变异(子代 2)
13. 将子代 1、子代 2 加入新种群
14. end while
15. 种群 ← 新种群
16. end for
17. best_path ← 对最优个体使用 BFS 连接相邻路径点
18. return best_path

3. 优缺点与改进方向

（1）优点：全局搜索能力强，不易陷入局部最优；不依赖梯度信息，可处理复杂约束。

（2）缺点：收敛速度慢，参数（种群规模、交叉率、变异率）调优困难；解的质量受编码方式影响大。

（3）改进方向：可采用自适应遗传算子（动态调整交叉/变异概率），或与局部搜索结合形成 memetic 算法。

2.4 强化学习算法

2.4.1 Q-Learning 与 SARSA

1. Q-Learning 原理

Q-Learning 是一种异策略时序差分学习算法。其更新规则为：

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]$$

其中 α 为学习率，γ 为折扣因子。Q-Learning 在更新时使用下一状态的最大 Q 值，与当前遵循的策略无关，这使得它可以学习到最优策略（贪心策略），即使探索时采用 ε-greedy 等随机策略。

2. SARSA 原理

SARSA 是一种同策略算法，其名称来源于更新所需元组 (s, a, r, s', a')。更新规则为：

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma Q(s',a') - Q(s,a)]$$

SARSA 在更新时使用实际执行的下一动作 a' 的 Q 值，因此其学习到的策略与探索策略

装 订 线

一致。与 Q-Learning 相比，SARSA 更为保守，倾向于规避高风险动作。

3. 探索策略

二者均采用 ε-greedy 策略平衡探索与利用：

（1）以概率 ε：随机选择动作

（2）以概率 1-ε：选择 argmax_a Q(s, a)

训练过程中，ε 逐渐衰减，使智能体从早期的大范围探索过渡到后期的策略利用。

4. 训练流程（以 Q-Learning 为例）

输入：环境 env，回合数 episodes，学习率 α，折扣因子 γ，探索率 ε

输出：Q 表

1. 初始化 Q 表为全零
2. for episode = 1 to episodes do
3. state ← env.reset()
4. done ← False
5. while not done do
6. 以 ε 概率随机选择 action，否则 action ← argmax Q[state]
7. next_state, reward, done ← env.step(action)
8. best_next ← max Q[next_state]
9. td_target ← reward + γ * best_next
10. Q[state, action] ← Q[state, action] + α * (td_target - Q[state, action])
11. state ← next_state
12. end while
13. 衰减 ε
14. end for
15. return Q 表

5. 算法比较

（1）Q-Learning：异策略方法，更新目标为 max Q(s', a')，倾向于学习最优策略，风险偏好较高。

（2）SARSA：同策略方法，更新目标为实际执行的 Q(s', a')，学习过程更保守，受探索策略影响较大。

6. 局限性

（1）维度灾难：表格型方法要求存储所有状态-动作对的 Q 值，状态空间增大时内存需求剧增。

（2）泛化能力差：表格法无法推广到未访问过的状态。

装 订 线

2.4.2 Double Q-Learning

1. 原理与动机

标准 Q-Learning 存在过高估计问题：由于更新目标 max_a' Q(s', a') 总是取最大值，即使 Q 值估计带有噪声，也会倾向于选择正偏置的动作，导致 Q 值被系统性高估。Double Q-Learning 通过维护两套独立的 Q 表 Q^A 和 Q^B 来解耦动作选择与价值评估：

（1）使用 Q^A 选择最优动作 a* = argmax_a Q^A(s', a)

（2）使用 Q^B 评估该动作的价值：Q^B(s', a*)

每次更新时随机选择更新 Q^A 或 Q^B，更新公式为：

$$Q_A(s,a) \leftarrow Q_A(s,a) + \alpha\left[r + \gamma Q_B(s', \arg\max_{a'} Q_A(s',a')) - Q_A(s,a)\right]$$

2. 算法流程

Double Q-Learning 的训练流程与 Q-Learning 相似，区别在于每次更新时随机选择更新 Q_1或 Q_2，并使用另一套 Q 表计算目标值。具体更新公式已在前文给出，此处不再赘述伪代码。

3. 优势

Double Q-Learning 有效降低了 Q 值的过高估计偏差，提升了策略学习的稳定性与最终策略质量。

2.4.3 深度 Q 网络与 DQN

1. 从表格到函数近似

当状态空间连续或离散但维度过高时，表格型 Q-Learning 不再适用。DQN 使用深度神经网络 Q(s, a; θ) 来近似动作价值函数，参数 θ 通过梯度下降优化损失函数：

$$L(\theta) = \mathrm{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(r + \gamma \max_{a'} Q(s',a';\theta^-) - Q(s,a;\theta)\right)^2\right]$$

其中 θ^- 为目标网络（Target Network）的参数，定期从策略网络 θ 复制。

2. 关键技术

（1）经验回放：将智能体与环境交互产生的转移元组 (s, a, r, s', done) 存储于回放池 D，训练时随机采样小批量样本，打破数据相关性，提高样本利用率。

（2）目标网络：使用独立的目标网络计算 TD 目标，每隔固定步数更新，稳定训练过程。

3. 网络结构

本文采用的 DQN 网络结构如下：

（1）输入层：2 个神经元（栅格坐标 x, y）

（2）隐藏层 1：128 个神经元，ReLU 激活

（3）隐藏层 2：128 个神经元，ReLU 激活

（4）输出层：4 个神经元，对应四个动作的 Q 值

4. 奖励塑形

为加速训练，本文在 DQN 中引入了奖励塑形技术。除原始环境奖励外，额外给予与目标曼哈顿距离减少量成正比的奖励：

装 订 线

$$r_{shaped} = r_{orig} + \beta \cdot (d(s, goal) - d(s', goal))$$

其中 β 为塑形系数（本实验取 0.2）。这种基于势能的奖励塑形不改变最优策略，但能提供密集的引导信号，显著加快收敛速度。

5. 训练流程

DQN 训练在 Q-Learning 基础上引入经验回放和目标网络：智能体与环境交互收集样本存入回放池，训练时随机采样批量样本，使用目标网络计算 TD 目标并更新策略网络，每隔固定步数将策略网络参数同步至目标网络。

6. 局限性

（1）训练成本高：DQN 需要大量交互样本和计算资源，训练时间远长于搜索算法。

（2）超参数敏感：学习率、网络结构、回放池大小、目标网络更新频率等均需精心调优。

（3）离散动作限制：DQN 原生仅支持离散动作空间，若需连续控制需改用 DDPG 等算法。

装 订 线

3 实验设计与实现

3.1 实验环境与工具

（1）编程语言： Python 3.8+

（2）核心库： NumPy（矩阵运算）、Matplotlib（可视化）、Gym（环境接口）、PyTorch（深度强化学习）等

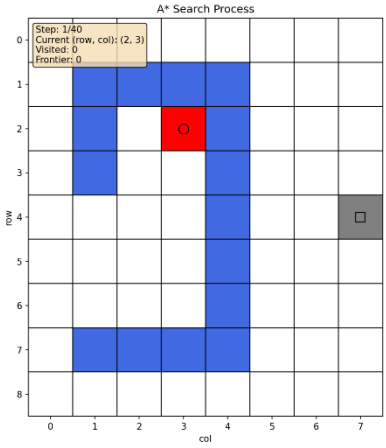
（3）运行平台： Windows 11

3.2 地图设计

为全面评估不同算法在多样化场景下的性能表现，本文设计了三类代表性的测试地图，其定义均位于 map_loader.py 文件中，通过 get_map(name) 函数按名称加载。

（1）原始地图（9×8）

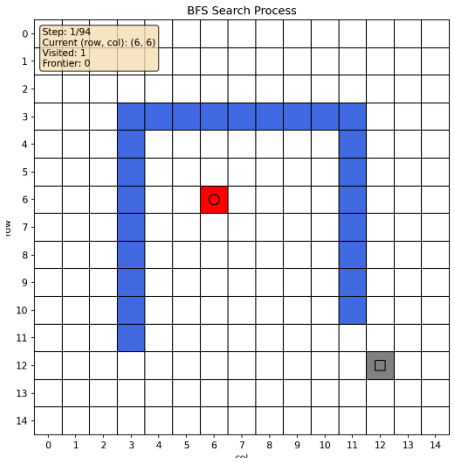
该地图障碍物呈“倒 L”形分布，在右侧留有一条宽度为 1 栅格的狭窄通道。起点位于左上方（2,3），终点位于右下方（4,7）。由于路径选择唯一且长度较短，该地图主要用于验证各算法能否在简单场景中正确找到最优路径，并对比其搜索效率。



（2）U 形陷阱地图（15×15）

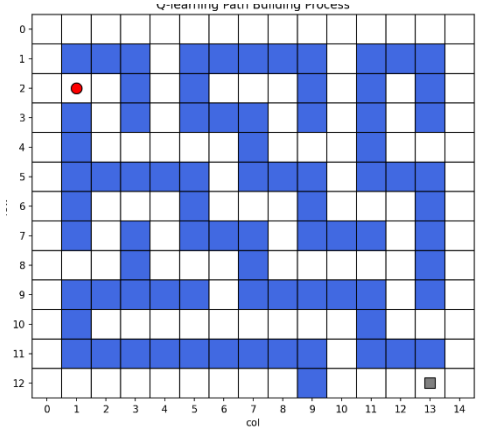
该地图设计了一个三面封闭、仅右下角开口的 U 形障碍区域。起点（6,6）位于 U 形障碍内部，终点（12,12）位于外部。此类结构对人工势场法和 Bug2 算法具有典型的“陷阱”效应——机器人容易被困于 U 形底部而无法沿梯度方向逃脱。该地图用于测试算法在非凸障碍环境下的全局寻优与局部脱困能力。

装 订 线



（3）自定义迷宫地图（13×15）

该地图包含多段不连续障碍墙与多个死胡同结构，整体呈迷宫状。起点（2,1）位于地图左侧中部，终点（12,13）位于右侧偏上位置。路径需多次绕行，且存在多个局部最优的“诱惑”分支。该地图用于综合考验算法在复杂环境中的全局搜索能力与收敛稳定性。



所有地图的起点和终点坐标均已在 map_loader.py 中预先设定，且保证位于自由栅格上，确保实验的一致性。

3.3 算法实现框架

为便于横向对比，本文为所有算法设计了统一的调用接口。每种路径规划算法均以一个独立的函数或类方法实现，其返回格式统一为：

path, frames = algorithm(grid, start, goal, ...)

其中：

（1）path：类型为 List[Tuple[int, int]]，表示从起点到终点的栅格坐标序列；若算法未能找到可行路径，则返回 None。

（2）frames：类型为 List[Tuple[Tuple[int, int], Set, Set]]，用于动画渲染的帧数据列表。每帧

装 订 线

包含当前节点坐标、已访问节点集合以及当前边界节点集合。

在工程实现上，本文针对部分算法的特有难题进行了针对性处理，以保证其公平地与基准算法比较：

（1）APF 的合力离散化：在每次迭代中，计算得到的合力向量 F_total 被映射到离其方向最近的四连通动作（上、下、左、右），使得输出路径严格满足栅格移动约束。该映射虽然可能牺牲一些平滑性，但确保了与搜索类算法路径的同基础对比。

（2）遗传算法的个体修复：在生成随机路径点时，强制其落在自由栅格上；交叉与变异后若产生无效片段（如穿越障碍物），会通过 BFS 局部重连的方式进行修复，避免适应度计算时的极端惩罚值影响选择压力。

（3）DQN 的奖励塑形实现：环境在执行每一步后返回原始奖励（到达目标为+10，碰撞为-1，其余为-0.1），在此基础上叠加基于曼哈顿距离减少量的塑形奖励，最终奖励作为训练信号，引导智能体更快地向目标靠近。

主程序 main.py 通过一个算法字典依次调用各算法，记录运行耗时与结果，并在每种算法执行完毕后自动调用可视化模块展示搜索动画。

3.4 可视化方案

可视化模块 visualize.py 提供了两个核心动画函数，分别适用于不同类型的算法：

（1）animate_search：用于经典搜索算法（BFS、A*、Dijkstra）。动画展示 closed 集（已扩展节点）、open 集（边界节点）以及当前正在扩展的节点，能够直观反映搜索边界逐层扩张的过程。

（2）animate_path_with_frontier：用于启发式、进化及强化学习算法。动画展示路径构建过程中已访问的节点和当前边界节点，重点呈现最终路径的生成轨迹。

这样区分设计的主要考量在于：搜索算法的价值主要体现在其搜索空间的扩张形态上，因此需要显式区分 open 与 closed 集；而 APF、GA 和 RL 等算法并非以“图搜索”方式扩展节点，它们的“搜索过程”更多表现为试探性路径的迭代优化或策略学习的轨迹，因此动画着重展现路径的生成与调整过程，使用户能够直观对比不同算法探索策略的本质差异。

动画采用 Matplotlib 的 FuncAnimation 实现，支持逐帧实时播放。地图中各类元素的颜色映射如下：

白色：自由栅格（可通行区域）

蓝色：障碍物栅格（不可通行）

浅蓝色：已访问/已扩展节点

金色：当前边界节点（位于开放集中）

红色：起点位置（以圆形标记）

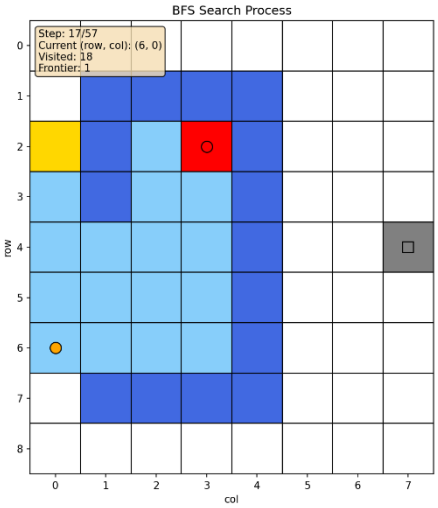
灰色：终点位置（以方形标记）

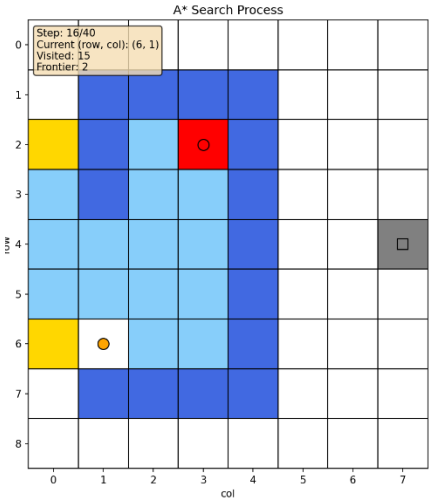
黄色：当前正在处理的节点（以高亮圆形标记）

装 订 线

最终路径在动画末尾以浅蓝色轨迹叠加显示，并附有路径长度与节点数量的文本统计信息。该可视化方案清晰地区分了搜索过程与最终结果，有助于直观理解不同算法的行为差异。

探索过程示例图片：（最终路径展示见下一部分）





装 订 线

4 实验结果与分析

本章对十种路径规划算法在三类测试地图上的运行结果进行定量对比与定性分析，实验记录了各算法的成功与否、路径长度、运行耗时及扩展节点数（或搜索步数）。

4.1 原始地图

原始地图（9×8）结构简单，起点与终点间仅有一条狭窄通道。在此地图上，经典搜索算法与强化学习算法均成功找到长度为 20 的最优路径。BFS 与 Dijkstra 扩展节点数均为 57，A* 借助曼哈顿启发式仅扩展 40 个节点，搜索效率提升约 30%。GA 收敛至最优解（扩展 21 步），但耗时显著高于搜索算法（4.345 s）。表格型强化学习方法（Q-Learning、SARSA、Double Q）均输出最优路径，规划耗时在 0.7~1.2 s 之间。DQN 同样成功，但训练耗时高达 325.985 s，性价比明显偏低。

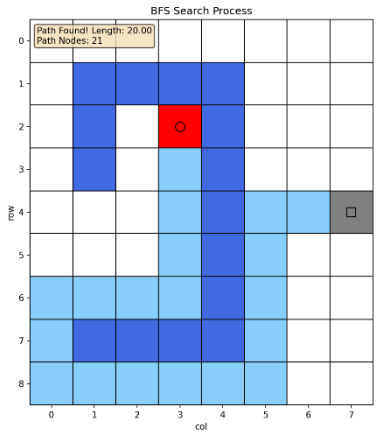
而 APF 和 Bug2 在此地图上的失败并非偶然。通过动画展示可以清晰观察到，APF 机器人在狭窄通道入口处反复震荡，表现为在几个栅格间来回跳动，直至触发震荡检测而退出；Bug2 则因为 M-线与障碍物近乎平行，在第一次遇障后沿墙跟随却始终找不到满足脱离条件的 M-线交点，最终耗尽步数。这两类失败模式正是其算法原理中固有缺陷的典型体现。

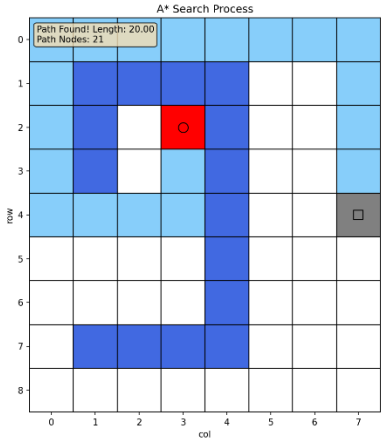
具体实验结果如下表所示：

算法	成功	路径长度	耗时/s	扩展节点数
BFS	✓	20	0.000	57
A*	✓	20	0.001	40
Dijkstra	✓	20	0.001	57
APF	✗	—	0.001	—
GA	✓	20	4.345	21
Bug2	✗	—	0.050	—
Q-Learning	✓	20	0.928	20
SARSA	✓	20	0.719	20
Double Q	✓	20	1.233	20
DQN	✓	20	325.985	20

装 订 线

各算法最终路径展示：（以 A*算法和 BFS 算法为例）





4.2 U 形陷阱地图

U 形陷阱地图（15×15）将起点置于三面封闭的 U 形障碍内部。BFS、A、Dijkstra 均以最优路径长度 12 成功，其中 A 扩展 39 个节点，远少于 BFS 的 94 个与 Dijkstra 的 92 个。GA 同样收敛至最优解（扩展 13 步），耗时约 5.656 s。表格型强化学习方法均成功输出最优路径，耗时在 0.5~1.1 s 之间。DQN 虽成功但耗时仍达 49.493 s。

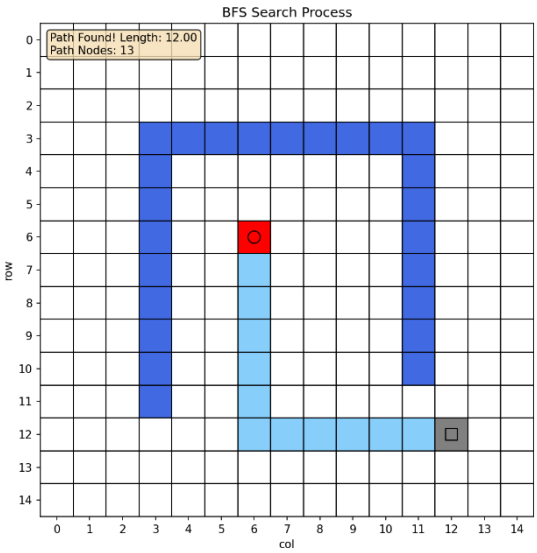
而 APF 在该地图的失败进一步印证了其在非凸障碍中的脆弱性。合力场在 U 形底部形成了一个明显的势能“盆地”，无论哪个方向移动都会使势能上升，标准的梯度下降法自然无法逃脱。Bug2 虽然成功，但其路径与最优解完全一致是这个特定地图的巧合，因为 M-线恰好穿越了 U 形障碍开口附近，使得机器人沿墙后迅速回到 M-线。若 M-线位置稍偏，Bug2 的路径长度可能远大于 12。这一现象恰好说明了 Bug2 的性能高度依赖于 M-线与障碍的相对几何关系。

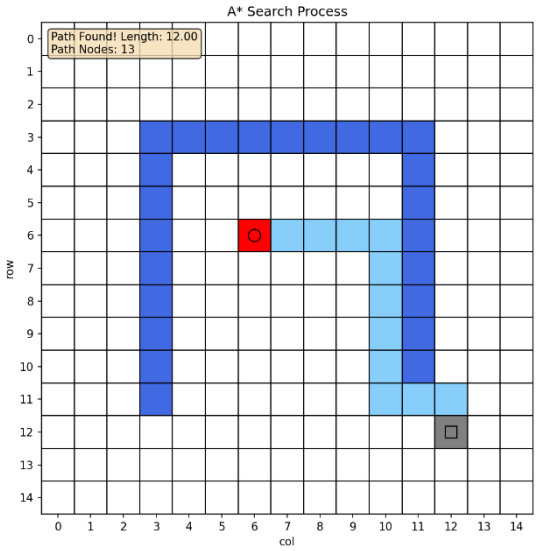
具体结果如下表所示：

算法	成功	路径长度	耗时/s	扩展节点数
BFS	✓	12	0.001	94
A*	✓	12	0.001	39
Dijkstra	✓	12	0.001	92
APF	✗	—	0.003	—
GA	✓	12	5.656	13
Bug2	✓	12	0.000	12
Q-Learning	✓	12	0.860	12
SARSA	✓	12	0.563	12
Double Q	✓	12	1.095	12
DQN	✓	12	49.493	12

各算法最终路径展示：（以 A*算法和 BFS 算法为例）

装 订 线





4.3 自定义迷宫地图

自定义迷宫（13×15）障碍密集、死胡同众多，对算法全局寻优能力提出较大考验。BFS 与 Dijkstra 均扩展 111 个节点找到最优路径（长度 24），A* 仅扩展 73 个节点，效率优势进一步扩大。Bug2 虽成功但路径长度高达 44，绕行严重，这是其沿墙策略在复杂边界上的典型表现。GA 输出路径长度为 28，略偏离最优解，耗时增至 11.475 s。表格型强化学习方法仍输出最优路径（长度 24），但训练耗时随地图复杂度增加而上升至 1.5~2.9 s。DQN 在该地图中输出次优路径（长度 30），且训练耗时高达 258.914 s，表现最不稳定。APF 在迷宫中多处陷入死胡同，宣告失败。

结合迷宫地图的路径可视化，可以更细致地分析算法行为。A*的路径行云流水，仅在少数关键岔路口扩展少量节点便迅速逼近目标，扩展的 73 个节点多集中在最优路径附近，几乎没有浪费。Bug2 的 44 步路径则在多处出现显著的“绕墙”痕迹，例如在左侧两个平行障碍之间反复贴边移动，即使已经错过了更直接的开口；GA 的 28 步路径虽然优于 Bug2，但在一个死胡同区域表现出轻微的“锯齿”绕行，显示出遗传算法可能在那个区域未能彻底优化路径点的走向。DQN 给出的路径长度为 30，比最优解长 6 步，观察动画发现它在两个决策点做出了“安全但绕远”的选择，这与 DQN 更倾向于保守策略、且在有限训练中尚未完全逼近最优 Q 值的特性是一致的。

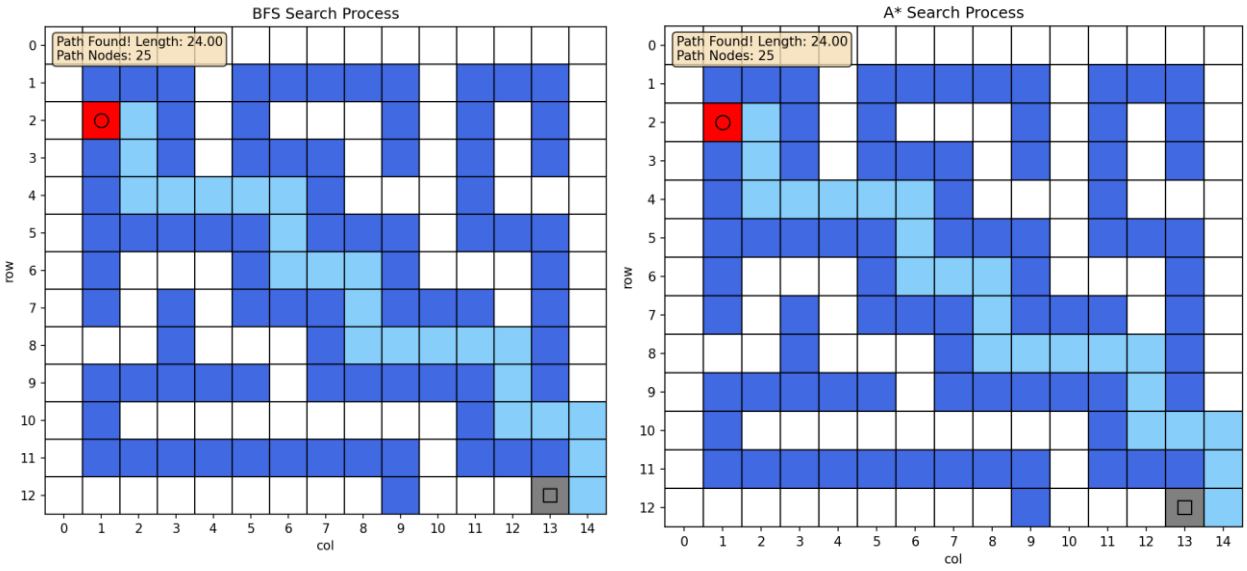
具体结果如下表所示：

算法	成功	路径长度	耗时/s	扩展节点数
BFS	✓	24	0.001	111
A*	✓	24	0.000	73
Dijkstra	✓	24	0.001	111

装 订 线

APF	✗	—	0.002	—
GA	✓	28	11.475	29
Bug2	✓	44	0.000	44
Q-Learning	✓	24	2.046	24
SARSA	✓	24	1.505	24
Double Q	✓	24	2.906	24
DQN	✓	30	258.914	30

各算法最终路径展示：（以 A*算法和 BFS 算法为例）



4.4 算法综合对比分析

综合三类地图的实验数据，可得出以下结论：

（1）成功率：经典搜索算法（BFS、A*、Dijkstra）在三类地图上均保持 100% 成功率，且始终输出最优路径。APF 在全部地图中均失败，验证了其在非凸障碍与狭窄环境中的脆弱性。Bug2 在原始地图中失败，在 U 形陷阱与迷宫中成功，但对复杂迷宫输出路径质量较差。GA 与强化学习算法在所有地图中均成功，但 GA 在迷宫中输出次优解，DQN 在迷宫中同样偏离最优。

（2）路径质量：A* 与 Dijkstra 始终给出最短路径。表格型强化学习方法在充分训练后也能稳定输出最优解。GA 与 Bug2 在复杂场景下路径长度明显增加，偏离最优解。DQN 在迷宫中仅得到次优结果，表明其在小规模离散任务中策略收敛不够精确。

（3）搜索效率（扩展节点数）：A* 的扩展节点数始终显著少于 BFS 与 Dijkstra，在迷宫中仅为后者的约 65%。强化学习方法在规划阶段的步数即为路径长度，无额外扩展，效率最高。GA 与 Bug2 的扩展步数与路径长度接近，但 Bug2 在迷宫中因大量绕行导致步数激增。

装 订 线

（4）时间开销：经典搜索算法耗时均在毫秒级，适合实时应用。表格型强化学习训练耗时在秒级（0.5~3 s），规划阶段极快。GA 随地图复杂度增加耗时明显上升（4~11 s）。DQN 训练时间最长（49~326 s），且随地图规模增大而急剧增长，在当前任务规模下性价比最低。

（5）适用场景：对于静态已知栅格地图，A* 算法以最优性、高效率和强鲁棒性成为首选。若需在线学习或环境动态变化，轻量级的表格型 Q-Learning 及其变体可作为备选方案。GA 适用于多目标约束优化场景，但需注意参数调优与收敛性问题。APF 与 Bug2 仅适合极简单场景。DQN 在小规模栅格规划中优势不明显，更适合处理高维连续状态空间的任务。

4.5 深入讨论

为进一步剖析算法行为上的差异，本节从启发式的信息价值、学习策略的收敛特性以及失败案例的根源三个角度进行深入讨论。

（1）启发式的信息价值量化：对比 A*、BFS 和 Dijkstra 在三张地图上的扩展节点数可以发现，地图越复杂，A*节省的节点比例越高。在原始地图中，A*扩展节点数约为 BFS 的 70%；在 U 形地图中约为 41%；而在迷宫地图中仅约 66%。这一趋势并非线性，原因在于 U 形陷阱地图中 A*的曼哈顿启发式与真实最短代价的误差极小，搜索方向几乎直指目标，因此扩展节点大幅减少；而迷宫地图虽复杂，但大量墙壁使曼哈顿距离与真实代价偏差增大，启发式引导效果有所削弱，因此节省比例有所回落。这直观地说明，A*的效率高度依赖于启发式的准确度。

（2）强化学习收敛过程对比：虽然表格中 Q-Learning 与 SARSA 最终均找到了最优路径，但通过观察训练日志可以发现，SARSA 的学习曲线（每回合步数）波动更小，较早收敛到稳定策略；而 Q-Learning 的曲线在训练前期震荡剧烈，个别回合步数极高，这与 Q-Learning 追求最优策略、在探索阶段更容易“犯大错”的特性相吻合。Double Q-Learning 则在二者之间取得了最好的平衡：既继承了 Q-Learning 的渐近最优性，又因为双估值器抑制了过高估计，使其学习曲线比标准 Q-Learning 更平稳，最终策略也更可靠。

（3）APF 与 Bug2 的失败根源再审视：APF 在三张地图上的全军覆没，根本原因在于其势场函数的“短视”本质——它仅根据局部梯度做决策，无法感知全局拓扑结构。即使是 U 形陷阱这样相对简单的非凸障碍，也会形成梯度上无法逃脱的局部极小。Bug2 在原始地图上的失败则与 M-线的选择强相关：M-线几乎平行于障碍物边界，导致“离开点”的几何条件从未被满足。这一现象提示，在实际应用中若不得不使用 Bug2，应考虑随机扰动 M-线方向或动态切换 M-线以增加成功概率。

装 订 线

5 结论和展望

5.1 结论

本文围绕静态栅格地图下的机器人避障寻径问题，设计并实现了一个集成十种路径规划算法的对比实验平台，涵盖经典搜索算法（BFS、Dijkstra、A*）、启发式与进化算法（APF、Bug2、GA）以及多种强化学习算法（Q-Learning、SARSA、Double Q-Learning、DQN）。在三类不同复杂度的测试地图上进行了系统性实验，从成功率、路径长度、搜索效率与时间开销等维度进行了定量对比与分析。主要结论如下：

（1）**A* 算法在静态全局规划中综合表现最优。**凭借曼哈顿启发式函数的有效引导，A* 在全部测试地图上均以 100% 成功率输出最短路径，且扩展节点数较 BFS 和 Dijkstra 大幅减少，运行时间维持在毫秒级。相比之下，BFS 与 Dijkstra 虽同样保证最优性与完备性，但因缺乏方向信息而扩展了大量无关节点，搜索效率明显偏低。

（2）**人工势场法与 Bug2 算法在非凸障碍环境中鲁棒性较差。**APF 在三类地图中全部失败，主要原因为 U 形陷阱与狭窄通道处引力与斥力相互抵消，触发了本文设计的震荡检测机制。Bug2 在原始地图中因 M-line 未能有效穿越障碍而失败，在复杂迷宫中虽成功但路径长度超过最优解长度，绕行严重。上述结果表明，此类算法仅适用于简单开阔场景，在实际复杂环境中需引入虚拟目标、随机扰动或与全局规划器协同使用以提高可靠性。

（3）**遗传算法具备全局寻优能力但存在收敛精度与速度的权衡。**GA 在简单及中等复杂度地图中能收敛至最优解，但在自定义迷宫中输出路径长度超过最优值，且运行耗时随地图规模增大而显著上升。其性能高度依赖种群规模、交叉变异率等参数设置，适用于对实时性要求不高的离线多目标优化场景。

（4）**表格型强化学习方法在小规模栅格任务中表现良好，DQN 优势不明显。**Q-Learning、SARSA 与 Double Q-Learning 经过充分训练后，在全部地图上均能稳定输出最优路径，规划阶段步数即为路径长度，效率极高，且训练耗时控制在 0.5~3 秒以内。Double Q-Learning 通过解耦动作选择与价值评估，在学习稳定性上略优于基础 Q-Learning。DQN 虽在简单地图中可收敛至最优解，但训练时间长达数十至数百秒，在复杂迷宫中仅得到次优路径，表明深度神经网络在此类离散小规模任务中性价比较低，更适合处理高维连续状态空间或基于视觉的导航任务。

（5）**可视化模块有效支撑了算法行为的直观理解。**通过动态展示已访问节点、边界节点及最终路径的生成过程，清晰呈现了盲目搜索与启发式搜索在扩展策略上的本质差异，以及强化学习智能体从随机探索到策略收敛的演变过程，为算法调试提供了直观工具。

5.2 展望

在本文工作的基础上，未来可从以下方向进一步研究与改进：

（1）**动态环境下的路径规划能力评估。**当前实验均基于静态已知栅格地图，而实际机器人作业环境往往存在移动障碍物或未知区域。后续可将环境扩展为部分可观测或动态变化场景，

装 订 线

评估各算法（尤其是强化学习方法）的实时重规划与在线学习能力。

（2）**深度强化学习方法的改进与应用拓展。**针对 DQN 在当前任务中收敛慢、稳定性差的问题，可尝试引入优先经验回放（Prioritized Experience Replay）、决斗网络结构（Dueling DQN）或近端策略优化（PPO）等更先进的深度强化学习算法，提升训练效率与策略质量。同时，可将输入状态由坐标扩展为局部栅格图像，验证端到端视觉导航策略的可行性。

（3）**多算法融合策略的探索。**鉴于单一算法各有优劣，可研究 A* 与 APF 的协同机制——利用 A* 规划全局最优路径作为参考线，引导 APF 进行局部动态避障，在保证全局最优性的同时增强对未知障碍的应对能力，避免陷入局部极小。

（4）**实验平台的交互化与工程化。**当前平台以脚本方式运行，后续可将其封装为具有图形用户界面的可交互应用程序，支持用户自定义地图、参数调节及算法选择性运行，并增加路径平滑、运动学约束等功能，使平台更贴近实际工程应用场景，便于教学演示与算法快速验证。

装 订 线

参考文献

[1] 蔡自兴. 机器人学基础[M]. 北京: 机械工业出版社, 2015.
[2] Dijkstra E W. A note on two problems in connexion with graphs[J]. Numerische Mathematik, 1959, 1(1): 269-271.
[3] Hart P E, Nilsson N J, Raphael B. A formal basis for the heuristic determination of minimum cost paths[J]. IEEE Transactions on Systems Science and Cybernetics, 1968, 4(2): 100-107.
[4] Khatib O. Real-time obstacle avoidance for manipulators and mobile robots[J]. The International Journal of Robotics Research, 1986, 5(1): 90-98.
[5] Lumelsky V J, Stepanov A A. Path-planning strategies for a point mobile automaton moving amidst unknown obstacles of arbitrary shape[J]. Algorithmica, 1987, 2(1): 403-430.
[6] Holland J H. Adaptation in natural and artificial systems[M]. Ann Arbor: University of Michigan Press, 1975.
[7] Watkins C J C H, Dayan P. Q-learning[J]. Machine Learning, 1992, 8(3-4): 279-292.
[8] Hasselt H V. Double Q-learning[C]// Advances in Neural Information Processing Systems. 2010: 2613-2621.
[9] Mnih V, Kavukcuoglu K, Silver D, et al. Human-level control through deep reinforcement learning[J]. Nature, 2015, 518(7540): 529-533.

装 订 线